Menu Driven Circular Singly Linked List Program

(Creation + Insertion)

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

/* Create Circular Linked List */
struct Node* create(struct Node *head)
{
    int n, i, value;
    struct Node *newnode, *temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        newnode = (struct Node*)malloc(sizeof(struct
Node));

        printf("Enter data: ");
        scanf("%d", &value);

        newnode->data = value;

        if(head == NULL)
        {
            head = newnode;
            newnode->next = head;   // circular link
            temp = newnode;
        }
        else
        {
            temp->next = newnode;
            newnode->next = head;   // maintain circular
link
            temp = newnode;
```

```
        }
    }
    return head;
}

/* Insert at Beginning */
struct Node* insertBegin(struct Node *head)
{
    struct Node *newnode, *temp;
    int value;

    newnode = (struct Node*)malloc(sizeof(struct Node));

    printf("Enter data: ");
    scanf("%d", &value);

    newnode->data = value;

    if(head == NULL)
    {
        head = newnode;
        newnode->next = head;
    }
    else
    {
        temp = head;

        while(temp->next != head)
        {
            temp = temp->next;
        }

        newnode->next = head;
        temp->next = newnode;
        head = newnode;
    }

    return head;
}

/* Insert at End */
struct Node* insertEnd(struct Node *head)
{
    struct Node *newnode, *temp;
    int value;

    newnode = (struct Node*)malloc(sizeof(struct Node));
```

```
    printf("Enter data: ");
    scanf("%d", &value);

    newnode->data = value;

    if(head == NULL)
    {
        head = newnode;
        newnode->next = head;
    }
    else
    {
        temp = head;

        while(temp->next != head)
        {
            temp = temp->next;
        }

        temp->next = newnode;
        newnode->next = head;
    }

    return head;
}

/* Insert at Specific Position */
struct Node* insertPosition(struct Node *head)
{
    struct Node *newnode, *temp;
    int value, pos, i;

    printf("Enter position: ");
    scanf("%d", &pos);

    printf("Enter data: ");
    scanf("%d", &value);

    newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = value;

    if(pos == 1)
    {
        return insertBegin(head);
    }
```

```
    temp = head;

    for(i = 1; i < pos - 1; i++)
    {
        temp = temp->next;
    }

    newnode->next = temp->next;
    temp->next = newnode;

    return head;
}

/* Display Circular Linked List */
void display(struct Node *head)
{
    struct Node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    printf("Circular Linked List: ");

    do
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while(temp != head);

    printf("(back to head)\n");
}

/* Main Function */
int main()
{
    struct Node *head = NULL;
    int choice;

    while(1)
    {
        printf("\n----- MENU -----\n");
        printf("1. Create\n");
```

```
        printf("2. Insert at Beginning\n");
        printf("3. Insert at End\n");
        printf("4. Insert at Position\n");
        printf("5. Display\n");
        printf("6. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                head = create(head);
                break;

            case 2:
                head = insertBegin(head);
                break;

            case 3:
                head = insertEnd(head);
                break;

            case 4:
                head = insertPosition(head);
                break;

            case 5:
                display(head);
                break;

            case 6:
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Example Run

```
----- MENU -----
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Position
5. Display
6. Exit

Enter choice: 1
Enter number of elements: 3
Enter data: 10
Enter data: 20
Enter data: 30

Enter choice: 5
Circular Linked List: 10 -> 20 -> 30 -> (back to
head)
```